

Actividad 5-18: Capturas - Typed Arrays

Desenvolupament WEB x JavaScript Typed Arrays x Web Storage API x Actividad_5-4_Carrito_de x Act_5-4_CarritoCompra x decimotava

w3schools.com/js/js_typed_arrays.asp

Search...

JAVASCRIPT SQL PYTHON JAVA PHP HOW TO W3.CSS C C++ C# BOOTSTRAP REACT MYSQL JQUERY EXCEL XML DJANGO

JavaScript Typed Arrays

Previous Next

Typed Arrays

Typed arrays was designed for handling **Binary Data**.

Unlike arrays, typed arrays are buffers of **Fixed Length**.

Typed arrays store elements of **Fixed Types** like 8-bit integers or 32-bit numbers.

Examples

Create a typed array of 5 bytes:

```
const myArr = new Uint8Array(5);
```

Try it Yourself »

Desenvolupament WEB en JavaScript | JavaScript Typed Array Methods | Actividad_5-4_Carrito_de_compra | Act_5-4_CarritoCompra.pdf | decimoctava - Buscar con Google

w3schools.com/js/js_typed_methods.asp

JavaScript | SQL | PYTHON | JAVA | PHP | HOW TO | W3.CSS | C | C++ | C# | BOOTSTRAP | REACT | MYSQL | JQUERY | EXCEL | XML | DJANGO

Typed Array Methods

[< Previous](#) [Next >](#)

The from() Method

The `from()` method creates a new typed array from any iterable object:

Examples

Create a typed array from a string:

```
const myArr = Int16Array.from("1234567890");
```

[Try it Yourself »](#)

Create a typed array from an array:

```
const myArr = Int16Array.from([1, 2, 3, 4, 5, 6, 7, 8, 9, 0]);
```

[Try it Yourself »](#)

Desenvolupament WEB en JavaScript | JavaScript Typed Array Reference | Actividad_5-4_Carrito_de_compra | Act_5-4_CarritoCompra.pdf | decimoctava - Buscar con Google

w3schools.com/js/js_typed_reference.asp

JavaScript | SQL | PYTHON | JAVA | PHP | HOW TO | W3.CSS | C | C++ | C# | BOOTSTRAP | REACT | MYSQL | JQUERY | EXCEL | XML | DJANGO

Typed Array Reference

[< Previous](#) [Next >](#)

Typed Array Types

Revised July 2025

Name	Range	Type
Int8Array	-128 / 127	1 byte signed integer
Uint8Array	0 / 255	1 byte unsigned integer
Uint8ClampedArray	0 / 255	1 byte unsigned integer
Int16Array	-32768 / 32767	2 bytes signed short
Uint16Array	0 / 65535	2 bytes unsigned short
Int32Array	-2^{31} / $2^{31} - 1$	4 bytes signed long

The screenshot shows the w3schools.com website with the URL `w3schools.com/js/js_arraybuffers.asp` in the address bar. The navigation bar includes links for JavaScript, SQL, PYTHON, JAVA, PHP, HOW TO, W3.CSS, C, C++, C#, BOOTSTRAP, REACT, MYSQL, JQUERY, EXCEL, XML, and DJANGO. The main heading is "JavaScript ArrayBuffer". Below the heading are "Previous" and "Next" buttons. A green box contains the text: "An **ArrayBuffer** is fixed a block of memory, often used to store **typed arrays**. On top of this block, you can create different **views** that interpret the bits as numbers, bytes, or other data types." Below this is the section "Creating an ArrayBuffer" with the text: "Use `new ArrayBuffer()` to create a new ArrayBuffer." An "Example" section follows with the text: "Create a 16-byte ArrayBuffer:".

JavaScript ArrayBuffer

[< Previous](#) [Next >](#)

An **ArrayBuffer** is fixed a block of memory, often used to store **typed arrays**.

On top of this block, you can create different **views** that interpret the bits as numbers, bytes, or other data types.

Creating an ArrayBuffer

Use `new ArrayBuffer()` to create a new ArrayBuffer.

Example

Create a 16-byte ArrayBuffer:

The screenshot shows the w3schools.com website with the URL `w3schools.com/js/js_dataview.asp` in the address bar. The navigation bar is the same as the previous page. The main heading is "JavaScript DataView". Below the heading are "Previous" and "Next" buttons. A green box contains the text: "The **DataView Object** lets you read and write multiple numeric types in an **ArrayBuffer**, at any byte offset, with optional control over **endianness** (byte order)." Below this is the section "What Is a DataView?" with the text: "A **DataView** is a **view** on top of an **ArrayBuffer**." Another text block states: "A **DataView** does not store data by itself; instead, it lets you interpret the bytes in the buffer as different types:" followed by a bulleted list: "8-bit, 16-bit, 32-bit integers (signed and unsigned)", "32-bit and 64-bit floating point numbers", "At arbitrary byte offsets in the buffer", and "With configurable **little-endian** or **big-endian** order".

JavaScript DataView

[< Previous](#) [Next >](#)

The **DataView Object** lets you read and write multiple numeric types in an **ArrayBuffer**, at any byte offset, with optional control over **endianness** (byte order).

What Is a DataView?

A **DataView** is a **view** on top of an **ArrayBuffer**.

A **DataView** does not store data by itself; instead, it lets you interpret the bytes in the buffer as different types:

- 8-bit, 16-bit, 32-bit integers (signed and unsigned)
- 32-bit and 64-bit floating point numbers
- At arbitrary byte offsets in the buffer
- With configurable **little-endian** or **big-endian** order

Use DataView when:

JavaScript Atomics

[< Previous](#) [Next >](#)

The Atomics Object

The **Atomics Object** provides low-level atomic operations on shared memory.

It is used with **SharedArrayBuffer** and **Typed Arrays** to share data between workers.

What Are Atomics?

When multiple threads (for example, the main thread and one or more workers) access the same data, you can get **race conditions**. Atomics helps avoid these race conditions by providing operations that:

- Work on shared typed arrays
- Are performed atomically (cannot be interrupted halfway)